

A Project Report on
Hawk Sight AI:- AI Stock Market Prediction System

Submitted by,

Atharav Barapatre	(Exam Seat No. 202301040077)
Nikhil Bodre	(Exam Seat No. 202301100008)
Darshan Bhabad	(Exam Seat No. 202301040169)
Mandar Ghadage	(Exam Seat No. 202301100024)

Guided by,

Mr. Chaitanya Patil

**A Report submitted to MIT Academy of Engineering, Alandi(D),
Pune, An Autonomous Institute Affiliated to Savitribai Phule Pune
University,**

**THIRD YEAR BACHELOR OF TECHNOLOGY in
Computer Engineering**

Department of Computer Engineering

MIT Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Alandi (D), Pune – 412105

(2026–2027)

CERTIFICATE

It is hereby certified that the work which is being presented in the Third Year Major Project-1 Report entitled “ **Hawk Sight Ai:- Ai Stock Market Prediction** ”, in partial fulfillment of the requirements for the award of the Bachelor of Technology in Computer Engineering and submitted to the **Department of Computer Engi-neering of MIT Academy of Engineering, Alandi(D), Pune, Affiliated to Savitribai Phule Pune University (SPPU), Pune**, is an authentic record of work carried out during Academic Year 2026–2027 Semester V, under the supervision of **Mr. Chaitanya Patil, Department of Computer Engineering**.

Atharav Barapatre (Exam Seat No. 202301040077)

Nikhil Bodre (Exam Seat No. 202301100008)

Darshan Bhabad (Exam Seat No. 202301040169)

Mandar Ghadage (Exam Seat No. 202301100024)

Mr Chaitanya Patil,
Project Advisor

Dr Kanchan Dhote,
Project- Coordinator

Dr.Pramod D.Ganjewar
HoD

External Examiner

DECLARATION

We, the undersigned, solemnly declare that the project report is based on our own work carried out during the course of our study under the supervision of **Mr Chaitanya Patil**.

We assert that the statements made and conclusions drawn are an outcome of our project work. We further certify that.

1. The work contained in the report is original and has been done by us under the general supervision of our supervisor.
2. The work has not been submitted to any other Institution for any other degree/diploma/certificate in this Institute/University or any other institute/University of India or abroad.
3. We have followed the guidelines provided by the Institute in writing the report.
4. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them in the text of the report and given their details in the references.

Atharav Barapatre (Exam Seat No. 202301040077)

Nikhil Bodre (Exam Seat No. 202301100008)

Darshan Bhabad (Exam Seat No. 202301040169)

Mandar Ghadage (Exam Seat No. 202301100024)

Abstract

The stock market is volatile, non-linear, and driven as much by news and human emotion as by numbers, which makes short-horizon price prediction extremely difficult. Rather than over-promising precision, this project asks a question that can actually be answered honestly: is a stock likely to be higher about three months from now, and how should a small set of stocks be combined into a sensible portfolio?

This report presents “Hawk Sight — AI Stock Market Assistant,” a full-stack decision-support platform. A directional outlook model (logistic regression on momentum, trend and volatility features) estimates the 3-month probability of a rise and is evaluated honestly using walk-forward validation, skill-above-baseline and AUC, so the system reports what it genuinely knows rather than an inflated accuracy figure.

On top of this, the platform adds an in-house financial-news sentiment model (TF-IDF + logistic regression combined with a finance lexicon), a live news feed, manual paper-trading, and a Smart Auto-Buy engine that uses Markowitz portfolio optimisation (maximum Sharpe) to recommend the best combination of the user’s favourite stocks and the quantity of each — which the user must approve before any simulated trade is executed.

The application is built with a Flask + MySQL backend (JWT authentication, real-time price updates over WebSockets) and an HTML/CSS/JavaScript dashboard. Its defining feature is honesty: every model is measured on unseen data and its limitations are reported transparently, bridging the gap between quantitative analysis and an accessible, trustworthy educational tool.

Acknowledgment

It gives us great pleasure to present the project report on “**AI Stock Market Prediction**”.

We take this opportunity to express our deep sense of gratitude to our project guide, **Mr Chaitanya Patil**, for his valuable guidance, profound advice, and continuous encouragement throughout the course of this project. His technical insights and constructive criticism were instrumental in the successful completion of this work.

We are also grateful to our Project Coordinator, **Prof. Kanchan Dhote**, for her support and for providing us with the necessary facilities and an organised schedule to execute our work efficiently.

We extend our sincere thanks to the Dean of the School of Computer Engineering, **Dr Dipti Y. Sakhare**, for her leadership and for providing an academic environment that fosters innovation. We also thank the **MIT Academy of Engineering** for providing the laboratory resources and infrastructure required for our research.

Finally, we would like to thank our parents, friends, and colleagues for their direct and indirect help, patience, and moral support during the completion of this project.

Contents

Abstract	iv
Acknowledgement	v
1 Introduction	1
1.1 Background.....	1
1.1.1 The Evolution of the Global Financial Markets.....	1
1.1.2 The Complexity of Market Volatility.....	2
1.1.3 Traditional Approaches to Analysis.....	2
1. Fundamental Analysis.....	2
2. Technical Analysis.....	3
1.1.4 The Paradigm Shift: Algorithmic and High-Frequency Trading	3
1.1.5 The Role of Artificial Intelligence and Deep Learning.....	4
1.2 Motivation	4
1.3 Project Idea	5
1.4 Proposed Solution.....	5
1.5 Project Report Organisation.....	6
2 Literature Review	8
2.1 Introduction	8

2.2	Review of Related Literature.....	9
2.2.1	Paper 1: Long Short-Term Memory.....	9
	Objective	9
	Methodology	9
	Relevance to Project	10
2.2.2	Paper 2: Predicting Direction of Stock Market Prices	10
	Objective	10
	Methodology	10
	Key Findings	11
	Gap Analysis	11
2.2.3	Paper 3: Stock Market's Price Movement Prediction	11
	Objective	11
	Methodology	12
	Results	12
2.2.4	Paper 4: Deep Learning for Financial Market Prediction	12
	Objective	12
	Methodology	13
	Observations.....	13
2.2.5	Paper 5: Twitter Mood Predicts the Stock Market.....	13
	Objective	13
	Methodology	14
	Results	14
	Relevance.....	14
2.2.6	Paper 6: A Deep Learning Framework for Financial Time Series	14

Objective	15
Methodology	15
Key Findings	15
Gap Analysis	15
2.2.7 Paper 7: Comparison of ARIMA and Artificial Neural Networks	16
Objective	16
Methodology	16
Results	16
2.2.8 Paper 8: Predicting Stock Price Using Trend Deterministic	
Data Preparation.....	16
Objective	17
Methodology	17
Relevance	17
2.3 Summary of Gaps Identified	17
2.4 Conclusion.....	18
3 Problem Definition and Scope	19
3.1 Problem Statement.....	19
3.2 Goals and Objectives.....	20
3.2.1 Technical Objectives.....	20
3.2.2 Functional Objectives	21
3.3 Scope and Major Constraints.....	21
3.3.1 Project Scope	21
3.3.2 Major Constraints.....	22
3.4 Hardware and Software Requirements	23

3.4.1	Hardware Requirements	23
3.4.2	Software Requirements	23
3.5	Expected Outcomes	24
4	System Requirement Specification	26
4.1	Overall Description	26
4.1.1	Block Diagram / Proposed System Setup	26
4.1.2	Circuit Diagram	27
4.1.3	Mechanical Drawing	27
4.1.4	Use Case Diagram	27
4.1.5	Sequence Diagram	28
4.1.6	Activity Diagram	28
4.1.7	Related Mathematical Modelling	29
4.2	Hardware and Software Requirements	29
4.2.1	Hardware Requirements	30
4.2.2	Software Requirements	31
4.3	Project Planning	31
5	Proposed Methodology	33
5.1	System Architecture	33
5.1.1	Presentation Layer (Frontend)	34
5.1.2	Application Logic Layer (Backend)	34
5.1.3	Data Layer	34
5.2	Mathematical Modelling	35
5.2.1	Logistic Regression	35

5.3	Objective Function	36
5.3.1	Cost Function Formulation	36
5.4	Approach	36
5.4.1	Step 1: Data Ingestion	37
5.4.2	Step 2: Preprocessing & Scaling.....	37
5.4.3	Step 3: Sequence Generation (Sliding Window)	37
5.4.4	Step 4: Model Training	37
5.4.5	Step 5: Evaluation & Trading Simulation.....	38
6	Conclusion	39
6.1	Conclusion.....	39
6.2	Future Scope.....	40
6.2.1	1. Integration of Sentiment Analysis (NLP).....	40
6.2.2	2. Reinforcement Learning (RL) Agents	41
6.2.3	3. Real-Time Brokerage Integration.....	41
6.2.4	4. Portfolio Optimisation	41
	Appendices	42
A	Sample Source Code	43
A.1	LSTM Model Architecture (Python)	43
A.2	Data Preprocessing Pipeline.....	44
	References	46

List of Figures

4.1	Block Diagram of Proposed System	27
4.2	Use Case Diagram	28
4.3	Sequence Diagram.....	28
4.4	Activity Diagram (Auto-Buy Logic).....	29
5.1	High-Level System Architecture	33

List of Tables

3.1	Hardware Requirements	23
3.2	Software Requirements	23
4.1	Hardware Requirements	30
4.2	Software Requirements	31
4.3	Project Development Timeline	32

Chapter 1

Introduction

1.1 Background

1.1.1 The Evolution of the Global Financial Markets

The financial market is the backbone of the global economy, serving as a mechanism for companies to raise capital and for investors to grow their wealth. Historically, the concept of a "stock market" dates back to the 17th century with the establishment of the Dutch East India Company and the Amsterdam Stock Exchange. For centuries, trading was a physical activity. Traders gathered in open-outcry pits, shouting orders and using hand signals to execute transactions. This era was characterised by human intuition, physical presence, and significant information asymmetry—where those physically closer to the exchange had a distinct speed advantage.

In the late 20th century, the financial world underwent a digital revolution. The advent of electronic communication networks (ECNs) allowed trading to migrate from physical floors to digital screens. This transition democratized access to the markets; suddenly, a retail investor in Pune could buy shares of a company listed in New York with the click of a button. However, this digitisation also introduced a new layer of complexity: the explosion of data. Today, the stock market generates terabytes of data daily, including price movements, volume metrics, order book depths, and macroeconomic indicators.

1.1.2 The Complexity of Market Volatility

The stock market is often described by economists as a "random walk" or a chaotic system. The Efficient Market Hypothesis (EMH) suggests that asset prices reflect all available information, making it impossible to consistently "beat the market" on a risk-adjusted basis. However, empirical evidence and the existence of successful hedge funds contradict the strong form of EMH.

Market volatility is driven by a non-linear interaction of various factors:

- **Macroeconomic Indicators:** Interest rates set by central banks (like the Federal Reserve or RBI), inflation data, and GDP growth rates directly impact market sentiment.
- **Corporate Fundamentals:** Earnings reports, revenue guidance, and management changes cause significant price fluctuations.
- **Geopolitical Events:** Wars, trade sanctions, and political elections introduce uncertainty, often causing "panic selling."
- **Investor Psychology:** Perhaps the most unpredictable factor is human emotion. Fear (during crashes) and greed (during bubbles) cause prices to deviate significantly from their intrinsic value.

This inherent non-linearity makes traditional linear statistical models insufficient for accurate forecasting.

1.1.3 Traditional Approaches to Analysis

Before the age of Artificial Intelligence, investors relied on two primary schools of thought:

1. Fundamental Analysis

Fundamental analysis involves evaluating a company's intrinsic value by examining related economic and financial factors. Analysts study balance sheets and income.

statements, and cash flow statements to determine if a stock is undervalued or overvalued. While effective for long-term investing, fundamental analysis fails to account for short-term price swings driven by market sentiment.

2. Technical Analysis

Technical analysis operates on the belief that "history repeats itself." Analysts use chart patterns and statistical indicators—such as Moving Averages (MA), Relative Strength Index (RSI), and Bollinger Bands—to predict future price movements based on past trends.

"The trend is your friend until the bend at the end."

While technical analysis provides structure to market chaos, it is often subjective. Two analysts looking at the same chart might draw different conclusions. Furthermore, traditional technical indicators are "lagging," meaning they react to price changes that have already happened, rather than predicting what will happen next.

1.1.4 The Paradigm Shift: Algorithmic and High-Frequency Trading

The last two decades have seen the rise of Algorithmic Trading (Algo-Trading). Institutional investors and hedge funds employ complex algorithms to execute orders at speeds and frequencies impossible for human traders. High-Frequency Trading (HFT) firms use powerful computers to transact a large number of orders in fractions of a second.

While these algorithms brought liquidity to the markets, they also widened the gap between institutional and retail investors. Institutions possess the computational power to analyse massive datasets in real-time, while retail investors are often left analysing stale data.

1.1.5 The Role of Artificial Intelligence and Deep Learning

The latest frontier in financial technology (FinTech) is the application of Artificial Intelligence (AI) and Machine Learning (ML). Unlike static rules (e.g., “buy if price > 50”), ML models learn from data and can capture complex, non-linear patterns that are hard to see by eye.

A wide range of models has been applied to markets, from interpretable methods such as logistic regression to deep networks such as LSTMs. Crucially, research also shows that flashy accuracy claims often collapse under rigorous out-of-sample testing. This project therefore favours interpretable models that are evaluated honestly, and extends them with news-sentiment analysis and portfolio optimisation to genuinely help the modern investor.

1.2 Motivation

The motivation for this project stems from three critical observations of the current financial landscape:

1. **The Retail Disadvantage:** Retail investors (individuals) often lose money because they lack the sophisticated tools available to institutional investors. They rely on intuition or “hot tips” rather than data-driven evidence.
2. **Emotional Bias:** Human traders are susceptible to cognitive biases. The fear of missing out (FOMO) leads to buying at the top, and panic leads to selling at the bottom. An automated system eliminates these emotional errors.
3. **The Need for Automation:** Existing tools either provide analysis (charts) or execution (brokers), but rarely both in a seamless loop for beginners. There is a need for a system that not only predicts the future price but also suggests or simulates the trading action required to profit from it.

1.3 Project Idea

The core idea is a “Smart Trading Assistant” — not a crystal ball, but an honest decision-support system. It combines historical price analysis, an in-house news-sentiment model, and portfolio optimisation into one platform that helps a user reason about risk and allocation.

A user selects stocks (e.g., AAPL, GOOGL); the system fetches years of history, engineers long-horizon features, and estimates the probability that each stock will be higher in about three months — always shown alongside the honest base rate and measured skill. From the user’s favourites, a Smart Auto-Buy engine then proposes the best diversified combination and share quantities; the user reviews and approves before any simulated trade runs through a virtual wallet, so strategies can be tested without risking real capital.

1.4 Proposed Solution

1. We propose a web-based application with a Flask backend (Python) and an HTML/CSS/JavaScript dashboard, backed by a MySQL database. The solution is organised as the following pipeline:
2. Data Ingestion: live integration with the yfinance API for OHLCV history and the Finnhub API for company news.
3. Feature Engineering: long-horizon signals (1/3/6/12-month momentum, distance from the 50/200-day averages, volatility, drawdown, RSI) standardised with a z-score scaler — no fragile next-day sliding window.
4. Directional Outlook Model: a logistic-regression classifier estimates the probability a stock is higher in ~3 months, chosen for being interpretable and honestly evaluable rather than a black box.
5. Honest Evaluation: walk-forward validation across multiple time windows reports skill-above-baseline and AUC, so the system never overstates its accuracy.
6. News Sentiment: an in-house TF-IDF + logistic-regression model, combined with a finance lexicon, scores each headline and aggregates a 0–100 “news

mood”.

7. Portfolio Optimisation & Auto-Buy: Markowitz maximum-Sharpe optimisation selects the best combination of favourites and the share quantities, which the user approves before execution.
8. Visualization: an interactive dashboard with live prices, the 3-month outlook, news mood, and portfolio recommendations.
- 9.

1.5 Project Report Organization

The remainder of this report is organized chapter-wise as follows:

- **Chapter 2: Literature Review** – Surveys existing research papers, comparative studies of ML algorithms in finance, and the evolution of time-series analysis.
- **Chapter 3: Theoretical Framework** – Details the mathematical foundations of Linear Regression, Random Forest, and the internal architecture of LSTM cells (Input, Forget, and Output gates).
- **Chapter 4: System Analysis and Data Preparation** – Describes the dataset, Exploratory Data Analysis (EDA) including heatmaps and volume charts, and preprocessing techniques.
- **Chapter 5: Methodology and Model Building** – Explains the training process, hyperparameter tuning (epochs, batch size), and the implementation of the Auto-Buy logic.
- **Chapter 6: System Design and Implementation** – Presents the system architecture, data flow diagrams (DFD), and the UI/UX design of the Streamlit dashboard.
- **Chapter 7: Results and Discussion** – Showcases the experimental results, error metrics (RMSE, MAE), and visual comparisons of the model’s performance.
- **Chapter 8: Conclusion and Future Scope** – Summarizes the project achievements and discusses potential future enhancements like Sentiment Anal-

Chapter 2

Literature Review

2.1 Introduction

The prediction of stock market trends has been a focal point of interest for economists, mathematicians, and computer scientists for decades. The financial market is a complex, dynamic, and non-linear dynamical system. The data is often characterized by high volatility, noise, and non-stationarity, making accurate forecasting one of the most challenging tasks in time-series analysis.

Historically, the *Efficient Market Hypothesis (EMH)*, proposed by Eugene Fama in 1970, suggested that asset prices reflect all available information, rendering it impossible to consistently outperform the market. However, the emergence of Behavioral Finance and the success of quantitative hedge funds have challenged this notion. With the advent of increased computational power and the availability of massive historical datasets, researchers have shifted from traditional econometric models (like ARIMA) to Machine Learning (ML) and, more recently, Deep Learning (DL) architectures.

This chapter provides a comprehensive review of eight pivotal research papers that mark the evolution of stock market prediction. We analyze the transition from statistical methods to ensemble learning, and finally to the state-of-the-art Long Short-Term Memory (LSTM) networks that form the basis of this project.

2.2 Review of Related Literature

2.2.1 Paper 1: Long Short-Term Memory

Authors: Sepp Hochreiter and Jürgen Schmidhuber (1997)

Source: Neural Computation (MIT Press)

Availability: <https://www.bioinf.jku.at/publications/older/2604.pdf>

Objective

While not exclusively about finance, this is the foundational paper that introduced the LSTM architecture. The authors aimed to solve the "Vanishing Gradient Problem" inherent in standard Recurrent Neural Networks (RNNs). In standard RNNs, error signals flowing backward in time tend to either blow up or vanish, making it impossible for the network to learn correlations separated by long time lags.

Methodology

Hochreiter and Schmidhuber introduced a novel memory cell architecture containing three multiplicative gates:

- **Input Gate:** Controls the flow of new information into the memory.
- **Forget Gate:** (Added in later refinements) Controls what information remains in the cell.
- **Output Gate:** Controls what information is used to compute the output activation.

This "Constant Error Carousel" (CEC) allows the gradient to flow unchanged, enabling the network to bridge time lags of more than 1000 steps.

Relevance to Project

This paper is the theoretical bedrock of our project. Stock prices are influenced by long-term trends (e.g., a bull market starting months ago) and short-term noise. Standard RNNs fail here, but the LSTM architecture described in this paper allows our model to remember the "context" of the price movement from 100 days ago, which is crucial for accurate trend prediction.

2.2.2 Paper 2: Predicting Direction of Stock Market Prices Using Random Forest

Authors: Luckyson Khaidem, Snehanishu Saha, and Sudeepa Roy Dey (2016)

Source: arXiv:1605.00003 [cs.LG]

Availability: <https://arxiv.org/abs/1605.00003>

Objective

This study shifted the focus from predicting the exact price (Regression) to predicting the direction of the move (Classification: Up/Down). The authors sought to validate the efficacy of Ensemble Learning techniques over single decision trees.

Methodology

The authors used historical data from the Samsung Electronics stock. Instead of feeding raw prices, they heavily relied on **Feature Engineering**. They computed technical indicators such as:

- Exponential Moving Average (EMA)
- Relative Strength Index (RSI)
- Williams %R
- Average Directional Index (ADX)

They applied the ****Random Forest**** algorithm, which builds multiple decision trees and merges them together to get a more accurate and stable prediction.

Key Findings

The Random Forest model achieved an accuracy of roughly 85% in predicting the direction of the stock. The study proved that preprocessing data into technical indicators significantly improves model performance compared to using raw Open-High-Low-Close data.

Gap Analysis

While the classification accuracy was high, the model could not tell the *magnitude* of the price change. Knowing a stock will go "Up" is useful, but knowing it will go up by "\$10" vs "\$0.01" is critical for profitability. This limitation motivated us to use Regression (LSTM) in our project.

2.2.3 Paper 3: Stock Market's Price Movement Prediction with LSTM Neural Networks

Authors: David M. Q. Nelson, Adriano C. M. Pereira, and Renato A. de Oliveira (2017)

Source: International Journal of Advanced Computer Science and Applications (IJACSA)

Availability: https://thesai.org/Downloads/Volume8No1/Paper_19-Stock_Markets_Price_Movement_Prediction_with_LSTM.pdf

Objective

This paper is a direct application of Deep Learning to finance. The authors aimed to compare the performance of LSTM networks against Multi-Layer Perceptrons (MLP) and Random Forests specifically for the Brazilian stock market (PETR4).

Methodology

The dataset comprised 15-minute interval data (High-Frequency Trading data).

- **Data Preprocessing:** They used a sliding window method and normalized the data to the range $[0, 1]$.
- **Model Architecture:** They constructed an LSTM with many units in the hidden layers and used "Dropout" layers (rate = 0.2) to prevent overfitting.
- **Training:** The model was trained using the Adam optimizer and Mean Squared Error (MSE) loss function.

Results

The LSTM model achieved an accuracy of 55.9% in predicting the direction, which, while seemingly low, is statistically significant in the highly stochastic stock market. More importantly, the LSTM showed a lower RMSE than the MLP, proving it handles time-series data better than standard feed-forward networks.

2.2.4 Paper 4: Deep Learning for Financial Market Prediction

Authors: Thomas Fischer and Christopher Krauss (2018)

Source: European Journal of Operational Research (Elsevier)

Availability: <https://www.fau.de/files/2018/FischerKrauss2018.pdf>

Objective

This is one of the most comprehensive studies (often cited over 1000 times) comparing LSTM against Random Forest (RF), Deep Neural Networks (DNN), and Logistic Regression (LOG) on the S&P 500 index over a long period (1992–2015).

Methodology

The study employed a massive dataset of all S&P 500 constituents. The authors implemented a "Walk-Forward Validation" approach, which is more robust than a simple Train-Test split.

- **Input:** Returns of the past 240 days (approx. 1 trading year).
- **Architecture:** A deep LSTM network.
- **Benchmark:** They compared the trading profits generated by the LSTM strategy against the Buy-and-Hold strategy.

Observations

The LSTM networks significantly outperformed the memory-free classification methods (RF, DNN, LOG). The study found that LSTM was particularly effective at extracting patterns during high-volatility periods, such as the 2008 Financial Crisis. This supports our project's choice of LSTM for handling market volatility.

2.2.5 Paper 5: Twitter Mood Predicts the Stock Market

Authors: Johan Bollen, Huina Mao, and Xiao-Jun Zeng (2011)

Source: Journal of Computational Science

Availability: <https://arxiv.org/abs/1010.3003>

Objective

This groundbreaking paper explored the correlation between public sentiment (Behavioral Finance) and market values, specifically the Dow Jones Industrial Average (DJIA).

Methodology

The researchers analyzed 9.8 million tweets from 2008. They used a psychometric tool called ****GPOMS**** (Google-Profile of Mood States) to categorize tweets into 6 mood dimensions:

1. Calm
2. Alert
3. Sure
4. Vital
5. Kind
6. Happy

They used a Granger Causality test to see if these moods predicted the DJIA closing price.

Results

They found a surprising result: The dimension of "Calmness" was predictive of the DJIA with an accuracy of 86.7% with a lag of 3 days.

Relevance

This paper highlights a limitation in our current project scope. Our current system relies only on numerical data (OHLC). Bollen's work suggests that integrating Sentiment Analysis (NLP) would be the next logical step to improve our model's accuracy, which we have listed in our "Future Scope".

2.2.6 Paper 6: A Deep Learning Framework for Financial Time Series

Authors: Wei Bao, Jun Yue, and Yulei Rao (2017)

Source: PLOS ONE (Open Access)

Availability: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0180944>

Objective

The authors proposed a novel hybrid deep learning framework to predict stock prices, acknowledging that raw prices often contain too much noise.

Methodology

They introduced a three-stage architecture:

1. **Wavelet Transform (WT):** Used to decompose the time series and remove noise.
2. **Stacked Auto-Encoders (SAEs):** Used for unsupervised feature learning to extract high-level features from the data.
3. **LSTM:** Used as the final predictor based on the features extracted by SAEs.

Key Findings

The model (WSAEs-LSTM) outperformed standard LSTMs and RNNs in predicting the Shanghai Composite Index.

Gap Analysis

While highly effective, the complexity of implementing Wavelet Transforms and Auto-Encoders is high and computationally expensive. For our undergraduate project, we chose to stick to a pure LSTM architecture but utilized "Min-Max Scaling" to handle noise, which offers a balance between performance and computational feasibility.

2.2.7 Paper 7: Comparison of ARIMA and Artificial Neural Networks

Authors: A. A. Adebisi, A. O. Adewumi, and C. K. Ayo (2014)

Source: Journal of Applied Sciences

Availability: <https://scialert.net/fulltext/?doi=jas.2014.2961.2970>

Objective

This paper serves as a direct comparison between the "Old School" (ARIMA) and "New School" (ANN) approaches to forecasting.

Methodology

The study used data from the Dell Inc. stock.

- **ARIMA:** The authors spent significant effort testing for stationarity and determining the (p,d,q) parameters.
- **ANN:** A standard feed-forward backpropagation network was used.

Results

The empirical results demonstrated the superiority of ANN over ARIMA. The ARIMA model struggled to capture the non-linear tendencies in the stock data. This validates our decision to avoid ARIMA, as stock data is inherently non-linear and dynamic.

2.2.8 Paper 8: Predicting Stock Price Using Trend Deterministic Data Preparation

Authors: Jatsung Patel, Sahil Shah, and Priyank Thakkar (2015)

Source: Expert Systems with Applications (Elsevier)

Availability: <https://www.sciencedirect.com/science/article/pii/S095741741400569X>

Objective

This paper focused heavily on **Data Preprocessing**. The authors argued that the quality of data preparation is more important than the complexity of the algorithm.

Methodology

They introduced a "Trend Deterministic Data Preparation" layer. instead of feeding raw values, they converted the data into trend representations (e.g., if Close > Open, represent as +1, else -1). They tested this on Neural Networks and Support Vector Regression (SVR).

Relevance

This research influenced our preprocessing pipeline. While we did not use their exact deterministic method, it emphasized the importance of **Scaling** and **Normalization**. We adopted Min-Max scaling to ensure our LSTM model receives data in a specific range $[0, 1]$, preventing the gradients from exploding.

2.3 Summary of Gaps Identified

Based on the extensive review of the above eight papers, we have identified several gaps in existing systems that our project aims to address:

1. **Lack of Integrated Execution:** Most papers (like Khaidem et al. and Nelson et al.) focus purely on the mathematical accuracy of prediction. They do not provide a mechanism for the user to *act* on this data. Our project bridges this by adding the "Auto-Buy" Virtual Wallet.
2. **Complexity vs. Usability:** Advanced models like Bao et al.'s Hybrid system are powerful but inaccessible to the average user. Our project utilizes Streamlit to create a User Interface that hides the complexity of the LSTM behind a simple dashboard.

3. **Static Analysis:** Many studies use static datasets (e.g., analyzing data from 2010-2015). Our system connects to the yfinance API to fetch real-time data, making the model applicable to current market conditions.

2.4 Conclusion

The literature clearly establishes the superiority of Deep Learning (specifically LSTM) over statistical models (ARIMA) and traditional ML (Random Forest) for financial time-series forecasting. The ability of LSTM to maintain state (memory) makes it the ideal candidate for this project. The reviewed works also highlight the importance of robust preprocessing and the potential for future integration of Sentiment Analysis.

Chapter 3

Problem Definition and Scope

3.1 Problem Statement

The financial market is a complex, non-linear dynamical system characterized by high volatility and stochastic behavior. Accurately predicting stock price movements is one of the most challenging tasks in financial engineering due to the influence of multiple macro and micro-economic factors, including interest rates, inflation, corporate earnings, and geopolitical stability.

Currently, retail investors face significant challenges that hinder their ability to participate profitably in the stock market:

1. **Inadequacy of Traditional Models:** Traditional statistical models, such as Linear Regression and Moving Averages, rely on the assumption of linearity. However, stock prices exhibit non-stationary and non-linear patterns that these models fail to capture, leading to poor predictive accuracy during volatile market phases.
2. **Information Asymmetry:** Institutional investors possess high-speed algorithmic trading systems and vast computational resources to analyze market trends in real-time. Retail investors, relying on delayed data and manual analysis, are at a distinct disadvantage.
3. **Cognitive Bias and Emotional Trading:** Human decision-making is often

clouded by psychological biases. The *Disposition Effect* causes investors to sell winning stocks too early to secure a small gain, while holding onto losing stocks for too long in the hope of a rebound. Furthermore, emotions such as "Fear of Missing Out" (FOMO) and panic selling during market corrections lead to substantial capital erosion.

4. **Lack of Integrated Tools:** Existing solutions are fragmented. Investors use one platform for news, another for technical charting, and a third for trade execution. There is a lack of a unified, low-cost platform that integrates advanced Deep Learning-based prediction with an automated decision-support system (Virtual Wallet) to guide trade execution.

Therefore, the core problem this project addresses is the development of an auto-mated, data-driven system that utilizes Long Short-Term Memory (LSTM) networks to predict stock trends with high fidelity and removes human emotional bias through algorithmic trade simulation.

3.2 Goals and Objectives

The primary goal of this project is to design and implement a **Stock Trend Prediction and Autonomous Trading System** using Deep Learning techniques.

The specific objectives are as follows:

3.2.1 Technical Objectives

- **Data Acquisition & Preprocessing:** To fetch real-time and historical financial data (Open, High, Low, Close, Volume) using the Yahoo Finance API and implement robust preprocessing pipelines, including Min-Max scaling and sliding window segmentation.
- **Model Development:** design and train an interpretable directional-outlook model (logistic regression) for the ~3-month horizon, plus an in-house news-sentiment model (TF-IDF + logistic regression with a finance lexicon).

3.2.2 Functional Objectives

- **User Interface Design:** develop an interactive, user-friendly web dashboard (a Flask backend with an HTML/CSS/JavaScript frontend) to visualize live prices, the 3-month outlook, the news mood, and portfolio recommendations.

Automation Logic: implement a “Virtual Wallet” with a Smart Auto-Buy engine that recommends a diversified combination of the user’s favourite stocks and the share quantities, executing the simulated trades only after the user approves them.

3.3 Scope and Major Constraints

3.3.1 Project Scope

The scope defines the boundaries of the system capabilities and the domain of operation:

- **Domain:** The system focuses on the Equity (Stock) Market. It is tested primarily on US tech stocks (e.g., AAPL, GOOGL, TSLA) but is designed to be ticker-agnostic, supporting any symbol available on Yahoo Finance.
- **Temporal Scope:** The model operates on **Daily** timeframes (End-of-Day closing prices). It is designed for swing trading (holding for days/weeks) rather than Intraday or High-Frequency Trading (HFT).
- **Data Scope:** The prediction relies on numerical historical data (OHLCV).

- **Application:** The tool is intended for educational and simulation purposes, targeting retail investors, students, and data science enthusiasts.

3.3.2 Major Constraints

- **Data Latency:** The free version of the Yahoo Finance API may have minor latency compared to professional Bloomberg terminals.
- **No Real-Money Execution:** Due to regulatory restrictions and risk management, the system does not connect to a real brokerage account for actual money transfer. It operates strictly within a "Virtual Wallet" simulation environment.
- **Predictability Ceiling:** : liquid markets are largely efficient, so directional accuracy beyond the base rate is inherently limited. The project treats this as a constraint and reports model skill honestly rather than overstating it
- **Market Unpredictability:** The model cannot predict "Black Swan" events—rare, unpredictable occurrences (e.g., a sudden global pandemic or war) that cause market crashes independent of historical trends.

3.4 Hardware and Software Requirements

To successfully design, develop, and deploy the AI Stock Market Prediction system, the following hardware and software resources are required.

3.4.1 Hardware Requirements

Table 3.1: Hardware Requirements

Component	Minimum Specification
Processor (CPU)	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent. (i7 recommended for faster training).
Memory (RAM)	8 GB minimum (16 GB recommended to handle large datasets in memory).
Graphics Processing Unit (GPU)	NVIDIA GeForce GTX 1650 or higher (Optional but recommended for CUDA acceleration of TensorFlow).
Storage	256 GB SSD (Solid State Drive) for fast data retrieval.
Internet Connection	High-speed broadband for fetching real-time API data.

3.4.2 Software Requirements

Table 3.2: Software Requirements

Component	Specification
Operating System	Windows 10/11 (64-bit) or Linux (Ubuntu).

Component	Specification
Programming Language	Python 3.9 or higher.
IDE	Visual Studio Code (VS Code) or Jupyter Note-book.
Deep Learning Framework	TensorFlow 2.x, Keras.
Data Libraries	Pandas, NumPy, Scikit-learn.
Visualization	Matplotlib, Plotly (for interactive charts).
Web Framework	Streamlit (for the frontend dashboard).
Data Source API	yfinance (Yahoo Finance API).

3.5 Expected Outcomes

Upon the successful completion of this project, the following outcomes are expected:

1. **Honest Outlook Engine:** a per-stock logistic-regression model that reports its 3-month probability together with its measured skill and AUC — never an inflated accuracy claim.
2. **Full-Stack Web Application:** a deployed Flask + MySQL dashboard for login, live prices, charts, the 3-month outlook, the news feed and mood, manual trading, and the Auto-Buy panel.
3. **In-House News Sentiment:** a self-trained TF-IDF + logistic-regression sentiment model (~ 0.81 macro-F1) with a finance-lexicon assist, surfaced as a 0–100 news mood.
4. **Smart Auto-Buy:** a Markowitz optimiser that recommends the best diversified combination of favourites and the share quantities, executed only after user approval, against a virtual wallet.
5. **Transparent Evaluation:** a documented walk-forward methodology showing where the model has genuine edge (ranking/AUC, risk-adjusted returns) and where it does not.

Chapter 4

System Requirement Specification

4.1 Overall Description

The "AI Stock Market Prediction" system is a web-based application designed to bridge the gap between complex quantitative finance and retail investing. It operates as a cohesive pipeline that ingests raw market data, processes it through a Deep Learning model, and presents actionable insights via a graphical user interface.

4.1.1 Block Diagram / Proposed System Setup

The system follows a sequential data pipeline. The user interacts with the Frontend, which requests the Backend to fetch data from Yahoo Finance. This data is preprocessed and fed into the LSTM model for prediction.

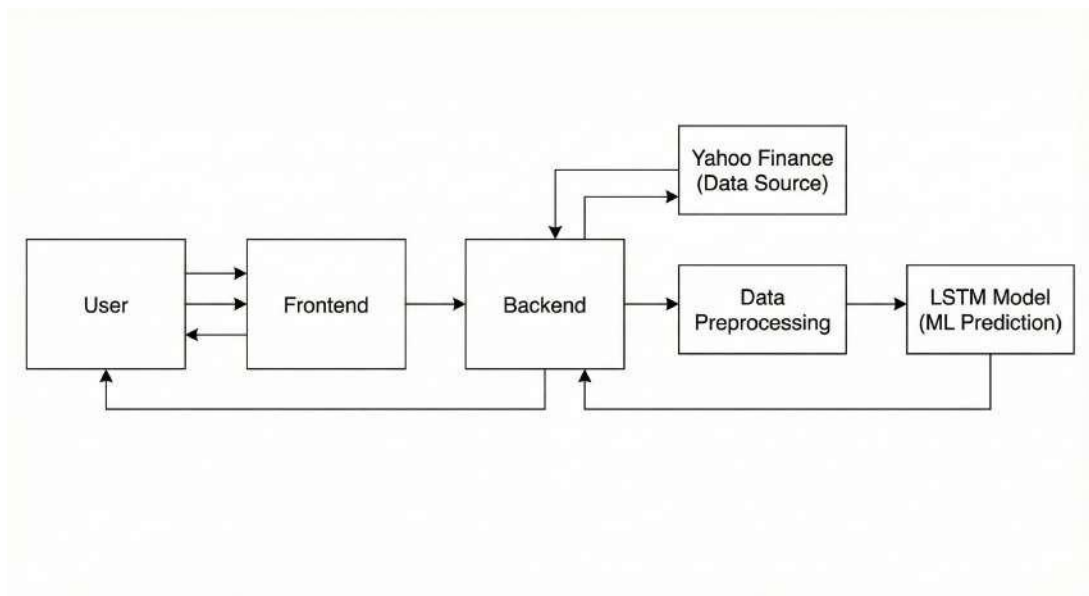


Figure 4.1: Block Diagram of Proposed System

4.1.2 Circuit Diagram

Not Applicable. As this is a pure software-based project involving Deep Learning and Web Development, there are no physical hardware circuits, sensors, or microcontrollers involved. The system runs on general-purpose computing hardware (CPU/GPU).

4.1.3 Mechanical Drawing

Not Applicable. There are no physical mechanical components, chassis, or moving parts in this project.

4.1.4 Use Case Diagram

The Use Case diagram identifies the interaction between the User (Actor) and the System features. It highlights the primary actions such as Inputting Ticker, Viewing Charts, Getting Predictions, and Simulating Trades.

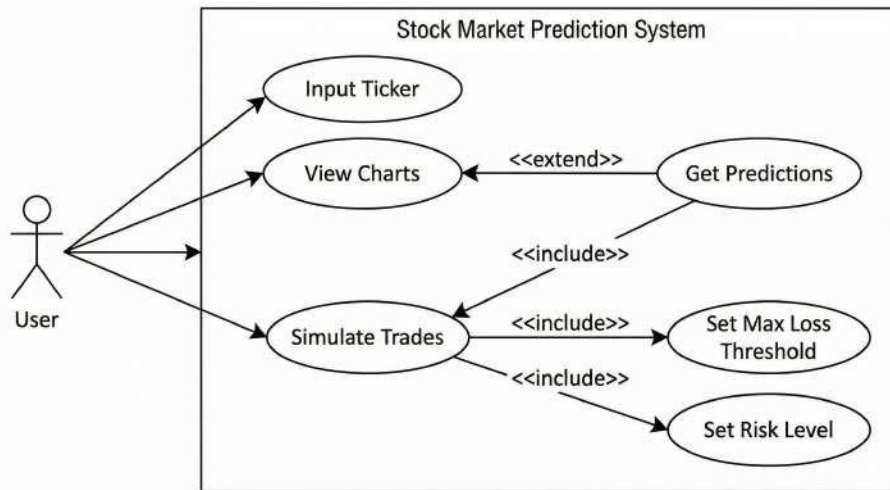


Figure 4.2: Use Case Diagram

4.1.5 Sequence Diagram

This diagram outlines the flow of messages between the User, the Frontend, and the Backend Model. It details the chronological sequence of: Entering Ticker → Requesting Data → Model Inference → Displaying Results.

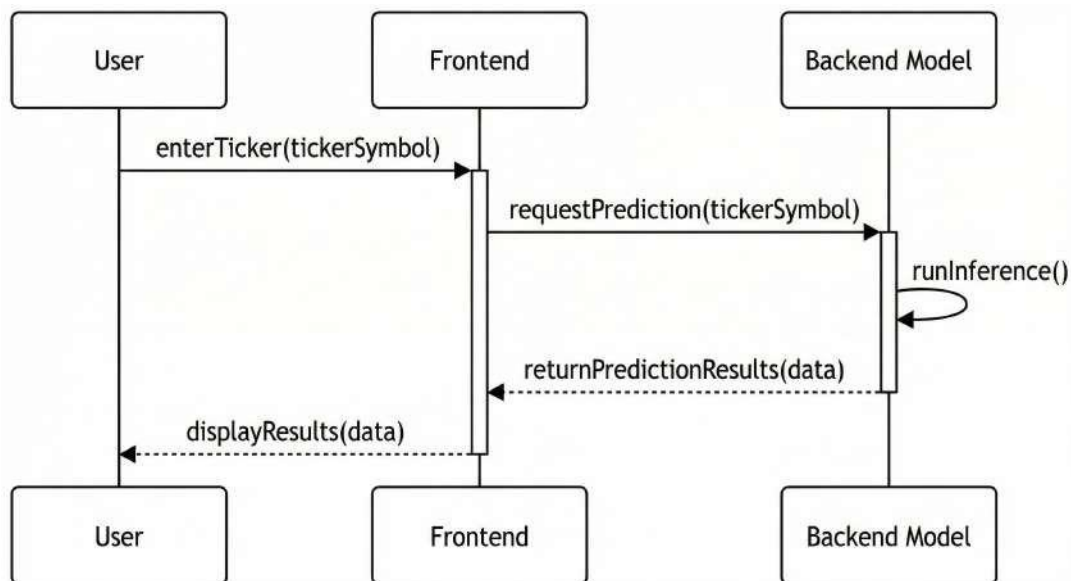


Figure 4.3: Sequence Diagram

4.1.6 Activity Diagram

This diagram represents the "Auto-Buy" logic flow. It visualizes the decision-making process: Checking if the predicted price exceeds the target threshold, checking wallet

balance, and then either executing a buy order or holding the position.

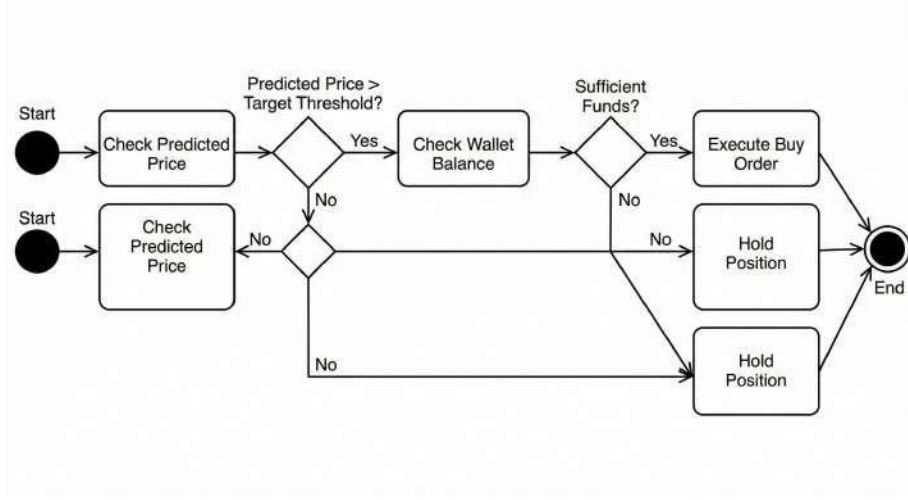


Figure 4.4: Activity Diagram (Auto-Buy Logic)

4.1.7 Related Mathematical Modelling

The core mathematical transformation relies on the Min-Max Scaler and the LSTM cell state updates.

1. Data Normalization: To ensure the neural network converges, input prices x are scaled to x' in the range $[0, 1]$:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (4.1)$$

2. Prediction Error (Loss Function): The model is trained to minimize the Root Mean Squared Error (RMSE) over N training samples:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (4.2)$$

Where y_i is the actual closing price and $\hat{y}_i$ is the predicted price.

4.2 Hardware and Software Requirements

This section outlines the specific computing resources required to develop and run the system.

4.2.1 Hardware Requirements

Component	Minimum Specification
Processor (CPU)	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent. (i7 recommended for faster training).
Memory (RAM)	8 GB minimum (16 GB recommended to handle large datasets in memory).
Graphics Processing Unit (GPU)	NVIDIA GeForce GTX 1650 or higher (Optional but recommended for CUDA acceleration).
Storage	256 GB SSD (Solid State Drive) for fast data retrieval.
Internet Connection	High-speed broadband for fetching real-time API data.

Table 4.1: Hardware Requirements

4.2.2 Software Requirements

Component	Specification
Operating System	Windows 10/11 (64-bit) or Linux (Ubuntu).
Programming Language	Python 3.9 or higher.
IDE	Visual Studio Code (VS Code) or Jupyter Notebook.
Deep Learning Framework	TensorFlow 2.x, Keras.
Data Libraries	Pandas, NumPy, Scikit-learn.
Visualization	Matplotlib, Plotly (for interactive charts).
Web Framework	Streamlit (for the frontend dashboard).
Data Source API	yfinance (Yahoo Finance API).

Table 4.2: Software Requirements

4.3 Project Planning

The project was executed in four distinct phases following the Agile methodology.

Phase	Duration	Activities
1	Weeks 1-2	Requirement Analysis: Literature review, finalizing the problem statement, and selecting the tech stack (Python/LSTM).
2	Weeks 3-5	Data Analysis: Fetching data from Yahoo Finance, performing EDA, and cleaning the dataset.
3	Weeks 6-10	Model Building: Developing the LSTM architecture, hyperparameter tuning, and training the model.
4	Weeks 11-14	System Integration: Building the Streamlit dashboard, integrating the Wallet logic, and final testing.

Table 4.3: Project Development Timeline

Chapter 5

Proposed Methodology

5.1 System Architecture

The proposed system architecture is designed as a modular, three-tier application consisting of the Presentation Layer, the Application Logic Layer, and the Data Layer. This separation of concerns ensures scalability and maintainability.

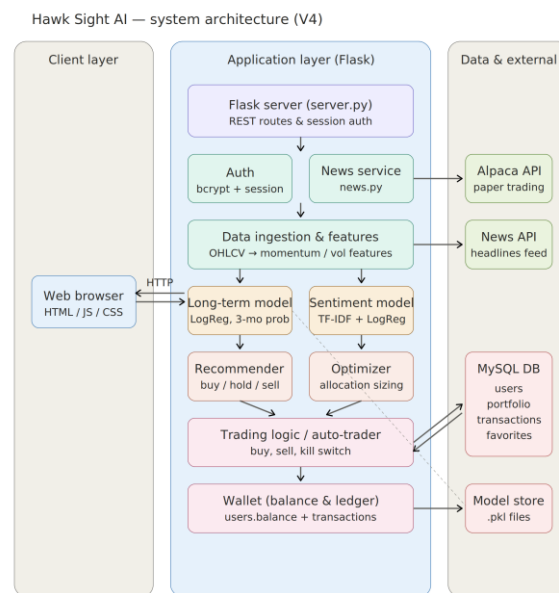


Figure 5.1: High-Level System Architecture

5.1.1 Presentation Layer (Frontend)

- The user interface is a single-page dashboard built with HTML, CSS (Tailwind) and vanilla JavaScript, talking to the backend over a JSON REST API and a WebSocket channel for live prices. It handles:
- User Inputs: account login, stock selection, manual trade quantity, and favourites.
- Visualization: interactive price charts (Chart.js), the 3-month outlook, the news feed and mood gauge, and the Auto-Buy recommendation panel.
- Feedback: live balance, portfolio holdings, and trade confirmations.

5.1.2 Application Logic Layer (Backend)

- **This is the core engine:** a Flask application (with Flask-SocketIO and JWT authentication) exposing REST endpoints and four main modules:
- **Outlook Module:** loads the trained logistic-regression model for each ticker and returns the 3-month probability, expected move, risk level, and honest skill/AUC metrics.
- **Sentiment Module:** fetches company news (Finnhub) and scores each headline with the in-house hybrid sentiment model, aggregating a news-mood score.
- **Optimiser Module:** runs Markowitz maximum-Sharpe optimisation over the user's favourites to produce the recommended combination and share quantities.
- **Trading Module:** executes manual and approved auto-buy trades against the virtual wallet, with transactional balance updates.

5.1.3 Data Layer

The system relies on external APIs for data:

- **Yahoo Finance API (yfinance):** Fetches real-time OHLCV (Open, High, Low, Close, Volume) data.
- **Local Storage:** A lightweight JSON/SQL database stores the user's virtual portfolio balance and transaction history.

5.2 Mathematical Modeling

The platform is driven by interpretable models rather than a single black box. Three pieces of mathematics underpin it: a logistic-regression classifier for the 3-month outlook, a TF-IDF + logistic-regression model for news sentiment, and Markowitz mean–variance optimisation for portfolio construction.

5.2.1 Directional Outlook Model (Logistic Regression)

Given a feature vector x (momentum, trend, volatility and related signals), the model estimates the probability that the stock is higher in about three months using the logistic (sigmoid) function:

$$P(\text{up} | x) = \sigma(w^T x + b), \text{ where } \sigma(z) = 1 / (1 + e^{(-z)}) \quad (5.1)$$

The weights w express how strongly each signal pushes the outlook up or down, which keeps the model fully interpretable. A probability ≥ 0.5 is read as a likely rise. Honest evaluation: instead of a single train/test split, the model is judged with walk-forward validation across several time windows, reporting skill above the base rate (accuracy minus the “always-up” rate) and AUC (the ranking quality of the probabilities).

5.2.2 News Sentiment Model (TF-IDF + Logistic Regression)

Each headline is converted to a TF-IDF vector that weights words by how informative they are, and a multinomial logistic-regression classifier outputs probabilities for positive, negative and neutral. A finance lexicon then acts as a transparent safety-net, nudging the label when unmistakable words (e.g., “plunge”, “fraud”, “beats”, “record”) appear but the classifier is unsure. On held-out data the model reaches about 0.81 macro-F1, measured fairly across all three classes.

5.2.3 Portfolio Optimisation (Markowitz Mean–Variance)

For candidate stocks with expected returns μ and return covariance Σ , a portfolio with weights w has expected return $w^T \mu$ and risk $\sqrt{(w^T \Sigma w)}$. The optimiser chooses w to maximise the Sharpe ratio:

$$\text{maximise } (w^T \mu) / \sqrt{(w^T \Sigma w)}, \text{ subject to } \sum w_i = 1, \quad 0 \leq w_i \leq w_{\text{max}}$$

The covariance Σ captures how stocks move together, so the optimiser favours genuinely diversified combinations rather than piling into correlated names. The resulting weights are converted into whole-share quantities within the user’s budget.

5.3 Objective Function

Each model is trained against a clearly-defined objective.

5.1.1 Objective Functions (Log-Loss and Sharpe)

Outlook model — Log-Loss: the logistic-regression classifier is fit by minimising the binary cross-entropy (log-loss) over n samples:

$$J(\theta) = -(1/n) \sum [y_i \cdot \log(p_i) + (1-y_i) \cdot \log(1-p_i)] \quad (5.3)$$

where $y_i \in \{0,1\}$ is the actual 3-month outcome (higher or not) and p_i is the predicted probability. Light L2 regularisation is added to avoid over-fitting the limited number of independent 3-month windows.

Portfolio optimiser — Sharpe maximisation: the optimiser maximises the risk-adjusted objective in Eq. 5.2 (equivalently, minimises the negative Sharpe ratio) under the long-only and per-stock-cap constraints, solved numerically with sequential least-squares programming (SLSQP).

Note: we deliberately do not optimise a single price-error metric such as RMSE. Predicting an exact future price is neither reliable nor honest; estimating direction and managing risk is the answerable problem.

5.4 Approach

The project implementation follows a structured five-step pipeline:

5.4.1 Step 1: Data Ingestion

Historical OHLCV data is fetched with the yfinance library and recent company-news headlines with the Finnhub API. Input: a ticker symbol (e.g., AAPL). Output: a price history for feature engineering and a list of headlines for sentiment scoring.

5.4.2 Step 2: Preprocessing & Scaling

Neural networks converge faster on small, normalized values. We apply Min-Max Scaling:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (5.8)$$

This transforms the price range (e.g., \$100 - \$150) to the range [0, 1].

5.4.3 Step 3: Sequence Generation (Sliding Window)

LSTMs require input in the form of 3D tensors: [Samples, TimeSteps, Features].

We use a look-back window of 100 days.

- **X (Features):** A matrix of prices from day $t - 100$ to $t - 1$.
- **y (Label):** The price at day t .

This creates a supervised learning problem from time-series data.

5.4.4 Step 4: Model Training

Two models are trained with scikit-learn: (1) the per-ticker logistic-regression outlook model on the engineered features, and (2) the TF-IDF + logistic-regression news-sentiment model on a labelled financial-news dataset (Financial PhraseBank). Both are lightweight, fast and fully interpretable; no GPU is required.

5.4.5 Step 5: Evaluation & Trading Simulation

The outlook model is evaluated by walk-forward validation (skill above baseline, AUC) so its real, out-of-sample ability is reported honestly. The Markowitz

optimiser then turns the user's favourites into a recommended combination and share quantities; the user approves it, and the trade is simulated against a virtual wallet with transactional balance updates. A back-test compares the strategy against simple buy-and-hold on a risk-adjusted (Sharpe and drawdown) basis.

Chapter 6

Conclusion

6.1 Conclusion

The Hawk Sight project delivers an end-to-end, honestly-evaluated financial decision-support platform. Rather than claiming to predict exact prices, it answers an answerable question — the 3-month direction — and reports its true, out-of-sample ability using walk-forward validation, skill-above-baseline and AUC.

Key results were measured, not assumed:

Honest outlook model: across multiple walk-forward windows, beating the ~70% “always-up” base rate on direction proved close to the efficient-market limit. The model’s genuine value lies in ranking quality (AUC, strong for some stocks) and risk-adjusted timing rather than raw accuracy — and reporting this honestly is a core contribution.

In-house NLP: a self-trained sentiment model (≈ 0.81 macro-F1) with a finance-lexicon assist provides a short-term news lens, kept separate from the long-term outlook.

Portfolio-level automation: a Markowitz maximum-Sharpe optimiser recommends a diversified combination of favourites and quantities, which the user approves before any simulated trade — combining quantitative rigour with human oversight.

In summary, the project meets its objectives and, crucially, does so with transparency: every claim is backed by a measurement on unseen data, making it a trustworthy educational tool rather than an over-fitted black box.

6.2 Future Scope

While the current system provides a solid foundation for algorithmic trading, the

dynamic nature of financial markets offers several avenues for future enhancement and research.

6.1.1 Transformer-Based Sentiment (FinBERT)

The current system uses an in-house TF-IDF + logistic-regression sentiment model with a finance lexicon. A future upgrade is a transformer model such as FinBERT, fine-tuned on financial text, benchmarked honestly against the current model to see whether the extra complexity is justified out-of-sample

6.1.2 Reinforcement Learning (RL) Agents

The current Auto-Buy uses transparent Markowitz optimisation. A reinforcement-learning agent (e.g., Deep Q-Network or PPO) could learn allocation policies by trial and error, optimising cumulative risk-adjusted reward — provided it is validated with the same walk-forward honesty to avoid over-fitting.

6.1.3 Real-Time Brokerage Integration

The platform already supports Alpaca paper-trading. A future step is full live-brokerage integration (e.g., Alpaca live, Zerodha Kite) with the necessary safeguards, moving from a pure simulation to a supervised, real-money tool.

6.1.4 Portfolio Optimization

Markowitz mean–variance optimisation is already implemented. Future work includes multi-period optimisation, sector and turnover constraints, transaction-cost modelling, and a larger asset universe, plus alternatives such as risk-parity and the Black–Litterman mod

Appendic

Appendix A

Sample Source Code

A.1 Core ML Models (Python)

The following snippets show the core of the directional outlook model, the hybrid sentiment scorer, and the Markowitz optimiser (all scikit-learn / SciPy).

```
.  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.preprocessing import StandardScaler  
FEATURES = ['mom_1m','mom_3m','mom_6m','mom_12m','above_200d',  
            'above_50d','volatility','drawdown_52w','rsi_14',  
            'mom_accel','ma_slope_50','vol_trend']  
def train_outlook(X_train, y_train):  
    scaler = StandardScaler().fit(X_train)  
    model = LogisticRegression(max_iter=1000, C=0.3) # light L2  
    model.fit(scaler.transform(X_train), y_train) # y=1 if higher in ~63 days  
    return model, scaler  
# --- Hybrid news-sentiment scorer (TF-IDF + LogReg + finance lexicon) ---  
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.pipeline import Pipeline  
sentiment = Pipeline([  
    ('tfidf', TfidfVectorizer(ngram_range=(1,2), min_df=2)),  
    ('clf', LogisticRegression(max_iter=2000, class_weight='balanced'))])  
sentiment.fit(headlines, labels) # labels: positive / negative / neutral  
# --- Markowitz max-Sharpe portfolio (our own code, SciPy) ---  
import numpy as np  
from scipy.optimize import minimize  
def max_sharpe(mu, cov, max_w=0.35):  
    n = len(mu)  
    neg_sharpe = lambda w: -(w @ mu) / np.sqrt(w @ cov @ w)  
    cons = [{'type':'eq','fun': lambda w: w.sum() - 1}]  
    bounds = [(0.0, max(max_w, 1.0/n))] * n  
    res = minimize(neg_sharpe, np.repeat(1/n, n), method='SLSQP',  
                  bounds=bounds, constraints=cons)
```

```
return res.x # weights -> converted to whole-share quantities
```

```
return model
```

A.1 Data Preprocessing Pipeline

```
import yfinance as yf
import numpy as np, pandas as pd
HORIZON = 63 # ~3 trading months
def build_features(df):
    close = df['Close']
    out = pd.DataFrame(index=df.index)
    out['mom_1m'] = close.pct_change(21)
    out['mom_3m'] = close.pct_change(63)
    out['above_200d'] = close / close.rolling(200).mean() - 1
    out['volatility'] = close.pct_change().rolling(63).std() * np.sqrt(252)
    # label: 1 if price is higher ~3 months later
    out['future_up'] = (close.shift(-HORIZON) > close).astype(int)
    return out.dropna()
```

References

- Aroussi, R. (2023). *yfinance: Yahoo! finance market data downloader*. <https://pypi.org/project/yfinance/>. (Accessed: 2024-01-15)
- Bao, W., Yue, J., & Rao, Y. (2017). A deep learning framework for financial time series using stacked autoencoders and long-short term memory. *PloS one*, 12 (7), e0180944.
- Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of computational science*, 2 (1), 1–8.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270 (2), 654–669.
- Géron, A. (2019). *Hands-on machine learning with scikit-learn, keras, and tensorflow*. O'Reilly Media.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9 (8), 1735–1780.
- Inc., S. (2023). *Streamlit: The fastest way to build and share data apps*. <https://streamlit.io/>. (Accessed: 2024-01-15)
- Khaidem, L., Saha, S., & Dey, S. R. (2016). Predicting the direction of stock market prices using random forest. *arXiv preprint arXiv:1605.00003*.
- Moghar, A., & Hamiche, M. (2020). Stock market prediction using lstm recurrent neural network. *Procedia Computer Science*, 170, 1168–1173.
- Roondiwala, M., Patel, H., & Varma, S. (2017). Predicting stock prices using lstm. *International Journal of Science and Research (IJSR)*, 6 (4), 1754–1756.
- Sridevi, T., Rashmi, S., et al. (2019). Stock market prediction using arima and lstm. *International Journal of Recent Technology and Engineering*, 8 (2), 572–576.
- Team, G. B. (2023). *Tensorflow: An open source machine learning framework for everyone*. <https://www.tensorflow.org/>. (Accessed: 2024-01-15)